■ SPACE COLONY

Object-Oriented Programming — Project Documentation

Android Application — Java / Android Studio

Course	Object-Oriented Programming
Platform	Android (minSdk 24 / targetSdk 34)
Language	Java
Build Tool	Gradle (Android Studio)
Submission	April 20, 2026

1. Project Description

Space Colony is a crew management Android application built in Java using Android Studio. The player manages a space station crew consisting of five specializations: **Pilot, Engineer, Medic, Scientist,** and **Soldier**. Each crew member has unique stats (skill, resilience, max energy) and a special ability bonus.

Core gameplay loop:

- **Recruit** crew members and assign specializations
- **Train** crew in the Simulator to gain experience (XP → Skill bonus)
- **Deploy** two crew members on cooperative turn-based missions
- **Survive** system-generated threats that scale with mission count
- **Recover** — defeated crew are sent to Medbay (No Death bonus feature)

The application is fully object-oriented, uses inheritance and polymorphism throughout the class hierarchy, and implements RecyclerView for all crew lists.

2. Implemented Features

2.1 Mandatory Requirements

✓	**OOP Principles**	Abstract CrewMember, inheritance (5 subclasses), polymorphism (act(), defend(), getColorResId(), getAvatarResId(), getSpecialAbilityDescription())
✓	**Code in English**	All source code, comments, Javadoc, and documentation written in English
✓	**Android / Java**	Targets Android API 24–34, built with Java in Android Studio Gradle project
✓	**Crew Management**	Recruit (Pilot/Engineer/Medic/Scientist/Soldier), place in Quarters, move to Simulator or Mission Control
✓	**Training System**	Simulator screen: +1 XP per session; effectiveSkill = baseSkill + experience
✓	**Cooperative Missions**	Two crew members vs. system-generated Threat; turn-based; retaliation; XP on win
✓	**Crew Recovery**	Return to Quarters restores energy; experience retained
✓	**Data Structures**	HashMap in Storage; ArrayList returned by getCrewByLocation()
✓	**Documentation**	This PDF; UML diagram; class descriptions; bonus feature list; AI disclaimer

2.2 Bonus Features Implemented

+1	**Crew Images**	Each specialization has a unique vector drawable avatar (ic_pilot, ic_engineer, ic_medic, ic_scientist, ic_soldier) displayed in every list row
+2	**Mission Visualization**	MissionActivity shows live energy ProgressBars for both crew members and the threat; bars update after every action; colour-coded per specialization
+2	**Tacti 6385cal Combat**	Player chooses ATTACK / DEFEND / HEAL each turn. DEFEND halves incoming damage (+3 resilience); HEAL (Medic only, once per mission) restores partner energy
+1	**Statistics**	StatisticsActivity shows colony-wide totals and per-crew records: missions completed, wins, training sessions
+1	**No Death**	Defeated crew are sent to MEDBAY instead of being deleted; they can be recovered (energy restored) and returned to Quarters
+1	**Randomness**	Threat.attack() adds random 0–2 bonus damage each turn via Random.nextInt(3), creating variance

+1	**RecyclerView**	All crew lists (Quarters, Simulator, Mission Control, Statistics) use RecyclerView with custom CrewAdapter / StatsAdapter

+2	**Specialization Bonuses**	Pilot +2 on NAVIGATION; Engineer +2 on REPAIR; Scientist +2 on ALIEN; Soldier +2 on COMBAT. MissionType chosen via Spinner in Mission Control

Total bonus points targeted: +11 (RecyclerView+1, Crew Images+1, Mission Visualization+2, Tactical Combat+2, Statistics+1, No Death+1, Randomness+1, Specialization Bonuses+2)

3. Class Architecture & OOP Design

3.1 Class Hierarchy

The application follows a strict OOP hierarchy. **CrewMember** is an abstract base class that defines the contract all specializations must fulfil through four abstract methods. Each of the five specializations (Pilot, Engineer, Medic, Scientist, Soldier) extends CrewMember and provides concrete implementations.

CrewMember	abstract	Base class. Holds all shared state (id, name, skill, resilience, experience, energy, location, statistics). Declares abstract methods act(), getColorResId(), getAvatarResId(), getSpecialAbilityDescription(). Provides concrete defend(), gainExperience(), getEffectiveSkill(), isAlive().
Pilot	extends CrewMember	skill=5, res=4, maxEnergy=20. Blue. actWithBonus() returns +2 on NAVIGATION missions.
Engineer	extends CrewMember	skill=6, res=3, maxEnergy=19. Yellow. actWithBonus() returns +2 on REPAIR missions. Wrench visual.
Medic	extends CrewMember	skill=7, res=2, maxEnergy=18. Green. healPartner() restores partner energy once per mission (Tactical HEAL action).
Scientist	extends CrewMember	skill=8, res=1, maxEnergy=17. Purple. actWithBonus() returns +2 on ALIEN missions.
Soldier	extends CrewMember	skill=9, res=0, maxEnergy=16. Red. Glass-cannon. actWithBonus() returns +2 on COMBAT missions.
Threat	standalone	System-generated enemy. Constructor scales stats: skill = 4 + missionCount, energy = 20 + missionCount×2. Random bonus 0–2 on each attack.
Storage	singleton	Central data store. HashMap indexed by crew id. Provides getCrewByLocation(Location) for all screens. Tracks totalMissions, totalRecruits.
MissionControl	service	Creates Threat instances; runs the cooperative mission algorithm; awards XP; handles No Death (Medbay routing). Static missionCount for difficulty scaling.
Quarters	service	Factory: createCrewMember() dispatches to correct subclass. Moves crew between locations. restoreEnergy() on return.
Simulator	service	train() calls gainExperience(1) and recordTrainingSession() on each selected crew member.
MissionResult	value object	Returned by MissionControl.launchMission(): boolean victory, List log, List survivorIds.

Location	enum	QUARTERS, SIMULATOR, MISSION_CONTROL, MEDBAY
MissionType	enum	NAVIGATION, REPAIR, ALIEN, COMBAT, HAZARD — drives threat names and specialization bonuses.

4. UML Class Diagram

The diagram below shows all model classes (UI Activity/Adapter classes excluded per grading criteria), their fields,methods,relationships.

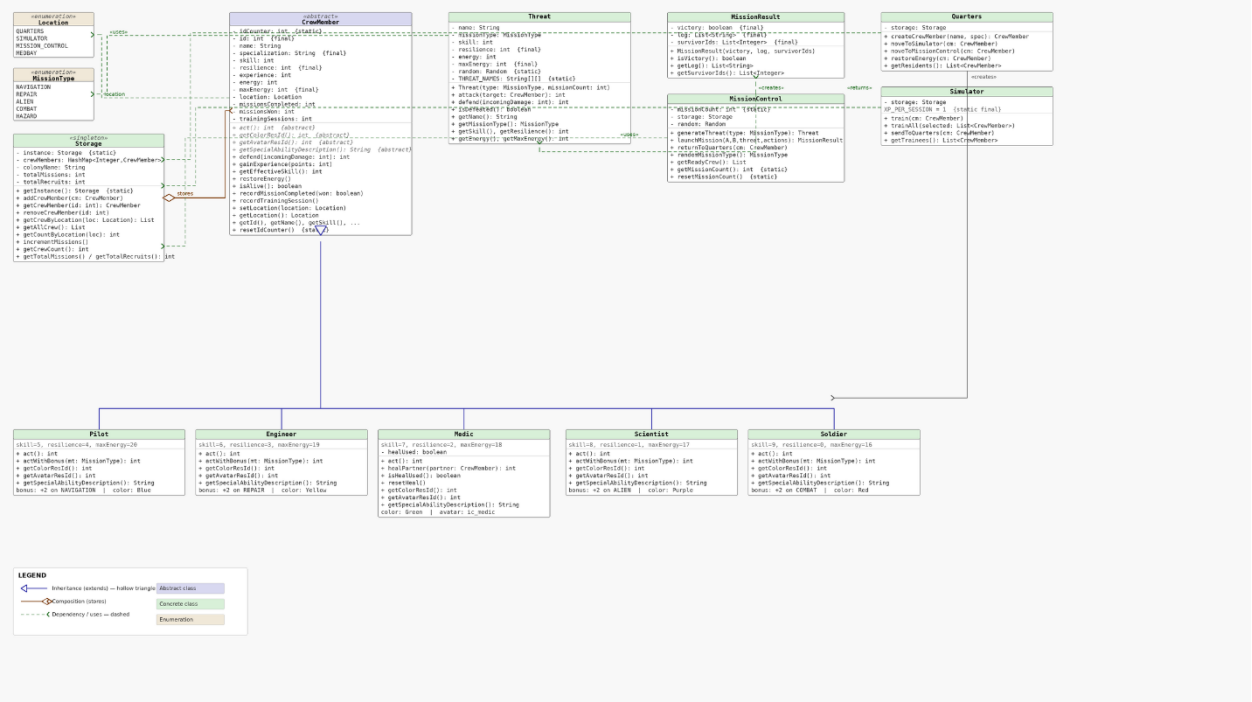

Figure 1: UML Class Diagram — Space Colony

Relationship key:

- **Solid arrow with hollow triangle** = Inheritance (extends)
- **Solid line with filled diamond** = Composition (Storage stores CrewMember instances)
- **Dashed arrow** = Dependency / Uses relationship.

5. Application Use-Flow

The user navigates between five screens from the main Colony Overview screen.

Screen	Purpose	Key Actions
Main / Colony Overview	Dashboard showing crew counts per location	Navigate to all other screens; view colony status
Recruit	Add new crew members	Enter name, select specialization (RadioButtons show stats preview), tap RECRUIT
Quarters	Manage resting crew	Select via checkbox; Move to Simulator or Mission Control; energy full here
Simulator	Train crew for XP	Select crew; TRAIN (+1 XP, +1 effective skill); or return to Quarters (energy restored)
Mission Control	Launch missions	Select 2 crew; pick MissionType from Spinner; LAUNCH MISSION → MissionActivity
Mission	Tactical battle	Per-turn: choose ATTACK / DEFEND / HEAL; watch energy bars; see scrollable log
Statistics	View performance	Colony totals; per-crew: missions, wins, training sessions

Medbay (implicit)	No-Death recovery	Defeated crew land here; move back to Quarters from Quarters screen to recover

6. Installation Instructions

Requirements: Android Studio Hedgehog (2023.1+) or newer; JDK 8+; Android SDK API 24–34.

Steps:

1. Clone or download the project repository from GitHub
2. Open Android Studio → *File* → *Open* → select the **SpaceColony** root folder
3. Wait for Gradle sync to complete (internet required for first sync to download dependencies)
4. Connect an Android device (API 24+) or start an AVD emulator
5. Press the **Run** ■ button or use *Run* → *Run 'app'*
6. The app will install and launch automatically

Dependencies (auto-downloaded by Gradle):

- androidx.appcompat:appcompat:1.6.1
- com.google.android.material:material:1.11.0
- androidx.recyclerview:recyclerview:1.3.2
- androidx.constraintlayout:constraintlayout:2.1.4
- androidx.cardview:cardview:1.0.0

7. Team Composition & Work Division

+

Student Name	Student ID	Contributions
[Bipro Das Bipu]	[002284413]	Model classes (CrewMember hierarchy, Threat, Storage, MissionControl); Mission algorithm; Tactical Combat
[Tafsir Ahmed]	[002364155]	UI Activities (MainActivity, QuartersActivity, SimulatorActivity); RecyclerView adapters; Crew Images drawables; Documentation
[Sameha Anan]	[002302065]	MissionActivity (live bars, logging); StatisticsActivity; XML layouts; Themes/Colors; Video recording

8. Tools Used

Android Studio	Hedgehog 2023.1 / newer	IDE, layout editor, AVD emulator
Java 8	JDK 8 (sourceCompatibility)	Programming language
Gradle	8.2	Build system, dependency management
Git / GitHub		Version control and submission repository
Material Design	1.11	UI components (CardView, TextInputLayout, ProgressBar, Spinner)
RecyclerView	1.3.2	Crew member lists with custom adapters
Vector Drawables	Android SDK	Specialization avatar icons

Github link and video link is in the online text section in moodle.